Start Here

Assignment 3, from zero

14 July 2026

Contents

1 Start here

You are about to be asked to explain this project to someone who knows more than you do. This document gets you from zero to being able to do that. Read it in order. It assumes you remember nothing.

Budget about two hours for the whole thing, including running the code.

1.1 1. What the assignment actually asked for

The teacher set one assignment with **two parts**.

Part A — population-based / swarm. Invent a problem where *one agent is too weak to solve it, but many weak agents working together can*. His words: put a single ant in a bowl and it just wanders. Put a colony together and they find food. Build something with that shape.

Part B — decision making. Invent a problem, model it as a Markov Decision Process, and solve it two ways: **Value Iteration** (which is handed the rules of the world) and **Q-Learning** (which is handed nothing and has to learn by trying things). Then say *in which setting you would use which one, and why*. That last bit is the actual assignment. The coding is the easy part and he said so out loud.

We built two separate problems, both set in a University of Dhaka hall, because he said separate problems are usually more convenient than forcing one problem to do both jobs.

1.2 2. The two problems, in plain language

1.2.1 Part A: where do you put the Wi-Fi routers?

A hall floor has 40 rooms and concrete walls. You can afford **3** access points. Where do you bolt them to the ceiling so that the most students get a usable signal?

You cannot just try every position, because position is a *continuous* thing — an AP can go at x = 12.3m or x = 12.31m or anywhere in between. There are infinitely many options. And you

cannot use calculus to ”roll downhill” to the best answer, because the walls make the signal jump in steps rather than change smoothly, so there is no hill to roll down.

So: **guess 30 layouts at random. Score them. Let them talk to each other and move towards the best one anybody has found. Repeat.** That is Particle Swarm Optimization. Each ”particle” is one complete guess at where all 3 APs go.

1.2.2 Part B: when do you switch on the water pump?

The same hall stores water in a tank on the roof, filled by an electric pump. Three things make this hard:

- **Load-shedding.** The power dies, and it dies *worst in the evening*, exactly when everyone wants a shower.
- **A diesel generator** can run the pump during an outage, but the hall buys only **2 hours of diesel per day**. Waste it in the morning and you have nothing at 9pm.
- **Demand spikes** morning and evening.

Every hour you choose: do nothing, pump using grid power, or burn diesel. The trick is that you should pump *before* the power goes out, filling the tank while electricity is still cheap and available. A rule like ”pump when the tank gets low” fails, because by the time the tank is low, the power is already gone.

1.3 3. The four ideas you need. That is genuinely all of them.

Fitness / reward. A number that says how good something is. In Part A, fitness = what percentage of students get signal. In Part B, reward = negative cost (running the pump costs money, students with no water costs a *lot*). Both algorithms are just machines for making that number bigger.

State. A snapshot of the world that is *enough* to decide what to do next. In Part B the state is `(how full the tank is, what hour it is, is the power on, how much diesel is left)`. That is 11 x 24 x 2 x 3 = **1,584** possible situations. If you know those four things, you do not need to know anything about the past. That property has a name — the **Markov property** — and it is the only reason either algorithm is allowed to work.

Policy. A rule that says, for *every* state, what to do. Not a plan (”pump at 3pm”), a *rule* (”if the tank is below 8 and the power is on, pump”). The output of Part B is a policy.

Model-based vs model-free. This is the whole point of Part B, so slow down here.

> A **model** is the rulebook: ”if the tank is at 5 and I pump, then 80% of the time it goes to 6, 20% of the time to 4,” and so on for every situation. If somebody hands you that rulebook, you can just *calculate* the best policy without ever touching reality. That is **Value Iteration** — it is model-based.
>
> If nobody gives you the rulebook, you have to *try things and see what happens*. That is **Q-Learning** — it is model-free. It pokes the world one hour at a time and slowly figures out what works.
>
> This is a difference in **what information you have**, not in how clever the algorithm is. Remember that sentence. It is the answer to half the questions he will ask.

1.4 4. Run it. Do not skip this.

```
cd 3
pip install -r requirements.txt

./run.sh swarm      # Part A, about 30 seconds
./run.sh rl         # Part B, about 4 minutes
```

Now open `results/plots/` and look at these three, in this order:

`spatial_swarm.png` — the hall floor. Red stars are the final AP positions. The orange lines are the swarm's best-guess *moving* over time. Watch how the APs spread out to get around the red walls. This is the picture that makes people understand Part A instantly.

`collective_behaviour.png` — the most important figure in the project. Every point on it costs the *same* amount of computing. One particle alone scores 76.8. Thirty particles that are forbidden from talking to each other score 87.3 — which is *no better than pure random guessing*. Thirty particles that share one number score 89.9.

> That is the teacher's ant-in-a-bowl point, proven. It is not the *population* that helps. It is the **communication**. If you only remember one result, remember this one.

`rl_policy_maps.png` — four grids. Left-right is the hour of the day, up-down is how full the tank is. Blue = pump on grid, red = burn diesel, grey = do nothing. Look at the top-right panel (the optimal policy when the power is out): the red region *grows* as you enter the shaded evening window. That is the agent *anticipating* — spending diesel more willingly at 7pm than at 7am, because it knows the outage will be long.

Then compare the bottom row (Q-learning) to the top row (Value Iteration). Same shape, but speckled and messy. That is what ”hasn't seen enough data yet” looks like.

1.5 5. The results, and the one that surprised us

Part A, everything at an identical budget of 3,030 attempts:

	score
1 particle, alone	76.8
30 particles, not allowed to communicate	87.3
(random guessing, for reference)	*87.4*
30 particles sharing one number	**89.9**

Part B, ranked by how far from perfect each method lands (”regret” — lower is better):

	what it knows	regret
Value Iteration, correct rulebook	everything	**0%**
Value Iteration, rulebook 25% wrong	almost everything	0.1%
Value Iteration on a rulebook *learned from data*	720,000 hours of experience	1.4%
Value Iteration, rulebook **4x** wrong	badly wrong	2.7%
A simple hand-written rule	nothing	14.5%
Q-Learning	720,000 hours of experience, no rulebook	**15.1%**
Doing whatever is cheapest right now	no future	120.6%

Read that table twice. We expected Q-Learning to win, because Dhaka's published load-shedding schedule is famously optimistic, so a planner using it should be badly misled. It did not win. A planner with a *four-times-wrong* rulebook still beat Q-Learning. And a rulebook we *learned from Q-Learning's own data* beat Q-Learning by a factor of ten.

So the honest conclusion is not ”model-free is better when the model is wrong.” It is:

> **If you can write down the rules of your world at all, do that and plan. A roughly-right model is worth more than 82 simulated years of trial and error. Model-free learning is what you reach for when you cannot write the rules down — not when they are merely wrong.**

That is a *negative* result about our own idea, and it is the strongest thing in the report. If he asks "so why did you bother with Q-Learning?", that paragraph is your answer.

1.6 6. Reading order for the docs

Do them in this order. Do not start with PART1, it will drown you.

0. **`PITCH.md`** — if the viva is *tomorrow*, start there instead. It is the run-of-show: what to say out loud, what to click, and every likely question answered.
1. **This file.** Done.
2. **`statement.md`** — the two problems written down formally, with the maths. Read it *after* you understand them informally, and the symbols will just be labels for things you already know.

   If the maths in it looks like a wall, open `MATH_EXPLAINED.md` beside it. That file takes every equation in `statement.md`, names every symbol in plain English, and then runs the formula with real numbers from our own hall so you can watch it compute. Start with its section 0 — a table of what the symbols are *said out loud as*. Most of what makes a formula frightening is notation, not ideas.
3. **`../report/main.pdf`** — the actual submission. Sections 5 and 6 are Part A and Part B. Read the "Results and Discussion" bits first; skip the literature review on a first pass.
4. **`PART3_viva.md`** — the question bank. This is what you rehearse from. It is long, and it is the single most useful file here the night before.
5. **`PART5_code_defense.md`** — line-by-line: "he points at the screen and asks *this*, you say *that*."
6. **`PART7_literature_applied.md`** — read this if he asks "did you actually read the papers?". It lists, paper by paper, which line of code changed because of it, and names the two papers that caught us claiming something false.
7. **`PART1_foundations.md`** (swarm theory) and **`PART6_decision_making.md`** (MDP theory) — the deep background. Read these only if you want to be able to go one level deeper than the question asked. They are reference material, not a tutorial.
8. **`PART4_problem_design.md`** — the problems we considered and rejected. Useful if he asks "why this problem and not something else?".

1.7 7. The five sentences you must be able to say without thinking

If you can say these five things cold, you will be fine.

1. *"A single particle scores 76.8, thirty particles that cannot communicate score 87.3, which is the same as random search, and thirty that share their best-found solution score 89.9. The population is not what helps — the communication is."*
2. *"The objective is not differentiable, because the signal drops in a step every time it crosses a concrete wall. So there is no gradient to follow, and gradient descent has nothing to work with."*
3. *"Value Iteration is model-based: it is handed P(s'|s,a) and it computes the answer. Q-Learning is model-free: it only ever sees one sampled transition at a time. That is a difference in information access, not in algorithm quality."*
4. *"We proved the problem genuinely needs lookahead: the formally-correct greedy policy — value iteration with gamma set to zero — loses by 121%. And at 2pm with a 4/10 tank, pumping costs 0.97 more that hour, but the optimal policy pumps anyway because it is worth +4.69 overall. Over five of those points come purely from the future."*
5. *"Our headline result is negative. A model that is wrong by a factor of four still beats Q-Learning trained on 720,000 steps, and a model learned from Q-Learning's own samples beats it by ten times. Model-free earns its place when you cannot write the model down at all — not when the model is merely wrong."*

1.8 8. What to do if he asks something you cannot answer

Say so, and say what you would do to find out. Every serious weakness in this project is already written down in the report's limitations section — the outage model is memoryless when real load-shedding is scheduled, we used discounted rather than average reward, and the Assignment 1 heuristic is admissible but loose. Naming your own weakness before he does is worth more than bluffing, and he will know the difference.